AWS Hands-On Project

Building an End-to-End Serverless Web App on AWS

Step-by-Step Architecture, Deployment, and Integration Tutorial

- **Topic:** Connecting core AWS services into a functional, real-world application.
- **Key Services:** AWS Amplify, AWS Lambda, Amazon API Gateway, Amazon DynamoDB, and AWS IAM.
- **Goal:** Build a fully operational math web application from scratch.

Project Architecture & Prerequisites

- **Prerequisites:** A text editor (Notepad/VS Code), an AWS Account with administrator access, and basic AWS familiarity.
- **Frontend Tier:** Static web page hosted and deployed globally using **AWS Amplify**.
- **Compute Tier:** Python-based serverless function using **AWS Lambda** for calculation logic.
- **API Layer:** REST API configured in **Amazon API Gateway** to securely trigger backend functions.
- **Storage Tier: Amazon DynamoDB** database storing historical results with fine-grained **IAM** permissions.

Frontend Hosting with AWS Amplify

- **Creating the UI:** Develop an `index.html` page containing input fields, labels, and action buttons.
- **Static Assets:** Package frontend code files into a clean deployment structure.
- **AWS Amplify Deployment:** Upload and host the web page instantly with automated CI/CD pipelines.
- **Public Access:** Provision a secure, public HTTPS URL to test and share your live web application immediately.

Serverless Compute: Python Lambda Functions

- **Function Setup:** Create a new serverless function in **AWS Lambda** using the Python runtime.
- **Business Logic:** Implement math calculation logic (e.g., exponent calculations) inside the handler function.
- **Event Handling:** Parse incoming JSON event payloads and return properly formatted HTTP/JSON responses.
- **Console Testing:** Use custom test event payloads to validate function execution directly in the AWS Console.

Exposing Compute via Amazon API Gateway

- **REST API Creation:** Set up a new REST API resource inside **Amazon API Gateway**.
- **Method Configuration:** Define HTTP methods (such as POST) to accept client requests from the frontend.
- **Lambda Integration:** Directly integrate API endpoints with the previously created AWS Lambda backend.
- **Stage Deployment:** Deploy the API to a live stage and obtain your unique public endpoint invocation URL.

Data Persistence & Security: DynamoDB & IAM

- **Database Provisioning:** Create a new **Amazon DynamoDB** table with a primary key schema.
- **IAM Policies:** Configure fine-grained execution roles granting Lambda permission (`PutItem`) to write to DynamoDB.
- **Code Integration:** Update the Lambda Python code using the `boto3` SDK to persist calculation results.
- **Verification:** Validate successful record insertion by checking table items after test invocations.

Integration & Cleanup

- **Frontend Wiring:** Update the JavaScript in `index.html` to call your API Gateway endpoint URL.
- **Re-Deployment:** Push the updated static files to **AWS Amplify** to complete the data loop.
- **End-to-End Test:** Input numbers in the web app, trigger the API, verify Lambda compute, and check DynamoDB storage.
- **Important Note:** Delete cloud resources (DynamoDB table, API Gateway, Lambda, Amplify apps) when finished to avoid charges.